

# integration\_sec01

## Chapter 1: Executive Summary & Integration Architecture

### 1.1 Integration Vision and Goals

The integration of LLMsVerifier into HelixTranslate represents a fundamental architectural transformation: the replacement of nine manually configured LLM providers with a unified, verification-driven provider management system that treats model quality as a first-class operational concern. This is not an incremental feature addition. It is a structural redefinition of how HelixTranslate discovers, validates, scores, and selects translation models. The central mandate is that **LLMsVerifier becomes the single source of truth** for all model availability, validation status, and performance ranking within the HelixTranslate ecosystem. No model reaches the user interface without passing through LLMsVerifier's eight-step verification pipeline. No model participates in translation without carrying a computed score from the five-component scoring engine. No provider remains invisible to the discovery system. These constraints are non-negotiable by design.

The current HelixTranslate architecture maintains provider integrations as discrete, hand-coded implementations scattered across `pkg/translator/llm/`. Each provider — whether OpenAI, Anthropic, Google Vertex, Azure OpenAI, Cohere, Mistral, Together AI, or local Ollama instances — exists as a standalone factory implementation that must be manually registered, individually configured with API keys, and separately maintained when endpoints or model catalogs change. This pattern, while functional for a small number of providers, creates compounding maintenance overhead as the LLM marketplace expands. Adding a tenth provider requires touching at minimum three files: the provider implementation, the factory registration, and the configuration schema. Adding a twentieth provider becomes a development sprint unto itself. The target architecture eliminates this friction entirely by delegating all provider lifecycle management to LLMsVerifier's automated discovery and verification pipeline.

Three UX guarantees govern this integration and serve as the definition of done for every implementation phase. **First: only validated models are available.** Any model that has not completed the full eight-step verification pipeline — covering API reachability, authentication, model existence, response format validation, latency measurement, capability detection, rate limit compliance, and error handling behavior — is invisible to the HelixTranslate runtime. There is no bypass mechanism, no emergency override, no administrator backdoor. **Second: only positively scored models participate in translation.** The five-component scoring

engine evaluates latency, throughput, cost efficiency, capability breadth, and reliability history. A model that passes verification but receives a composite score below the configurable threshold is gated from selection. **Third: automatic discovery of all supported providers.** When a new provider appears in the LLMsVerifier registry, or when an existing provider adds new models, HelixTranslate learns of them without code changes, configuration edits, or deployments. The discovery tier system — primary, secondary, and tertiary — polls provider registries, public model databases, and community endpoints on configurable intervals, feeding a continuously updated model catalog that the translation engine consumes in real time.

These guarantees translate directly into operational outcomes. Translation quality improves because only proven-capable models serve requests. Operational costs decrease because cost-efficiency scoring automatically prefer low-price-per-token models for appropriate workloads. Incident recovery accelerates because the health-check layer automatically removes degraded providers from rotation without human intervention. Development velocity increases because adding provider support requires zero code changes in HelixTranslate proper.

## 1.2 Reference Architecture from HelixAgent

The integration blueprint is drawn directly from LLMsVerifier's production implementation in the HelixAgent project, where the eight-phase startup pipeline has been running in production environments for multiple release cycles. Understanding this pipeline is prerequisite to mapping its components correctly into HelixTranslate's codebase. The pipeline, implemented in `internal/verifier/startup.go`, executes sequentially during HelixAgent process initialization and establishes the complete verification and scoring infrastructure before any translation request is accepted.

The pipeline begins with **Phase 1: Config Load**. The `LoadVerifierConfig()` function reads `configs/verifier.yaml` and unmarshals it into the `VerifierConfig` struct defined in `internal/verifier/config.go`. This configuration specifies provider API keys, discovery intervals, verification timeout thresholds, scoring weights, health check frequencies, and gating thresholds. The configuration is validated against a JSON Schema embedded at `internal/verifier/config/schema.json` — any missing required field or type mismatch causes a fatal startup error. This strict validation ensures that the verification pipeline never operates with incomplete or ambiguous configuration.

**Phase 2: StartupVerifier Initialization** instantiates the `StartupVerifier` struct, the central coordinator for all subsequent phases. This struct maintains references to the discovery service, verification pipeline, scoring engine, model registry, and health monitor. It also owns a `sync.WaitGroup` that gates server startup until all asynchronous initialization completes. The `StartupVerifier.Init()` method allocates these subsystems but does not yet execute them — that sequencing is controlled by the subsequent phases.

**Phase 3: Provider Discovery** triggers the three-tier discovery system implemented in `internal/verifier/discovery.go`. The **primary tier** queries well-known provider endpoints — OpenAI's `/models` endpoint, Anthropic's model listing API, Google's model catalog — using the stored API keys from configuration. The **secondary tier** scans public model registries including Hugging Face's API,

Replicate's model listing, and Together AI's model catalog. The **tertiary tier** probes community-discovered endpoints and user-configured custom providers. Each tier runs in its own goroutine and writes discovered models into a thread-safe `DiscoveryResult` channel. Discovery respects rate limits through a token bucket implemented in `internal/verifier/ratelimit/bucket.go`.

**Phase 4: Provider Verification** executes the eight-step verification pipeline for every discovered model, implemented in `internal/verifier/verification.go`. The eight steps are: (1) `CheckAPIReachability` — confirms the provider endpoint responds to HTTP requests within the configured timeout; (2) `CheckAuthentication` — validates that the stored API key has necessary permissions; (3) `CheckModelExistence` — confirms the specific model ID is available on the provider; (4) `ValidateResponseFormat` — sends a minimal prompt and confirms the response matches expected schema; (5) `MeasureLatency` — records p50, p95, and p99 response times over 10 warmup requests; (6) `DetectCapabilities` — identifies supported features (function calling, JSON mode, vision, streaming, tool use); (7) `CheckRateLimits` — determines request-per-minute and token-per-minute ceilings; (8) `ValidateErrorHandling` — confirms graceful error responses for malformed requests and rate limit exceeded scenarios. Each step produces a `VerificationStepResult` that feeds into the aggregate `ProviderVerification` record. A single step failure does not abort the pipeline — remaining steps continue so that the scoring engine can operate on partial data where appropriate.

**Phase 5: Provider Scoring** invokes the five-component scoring engine defined in `internal/verifier/scoring.go`. Each model receives five component scores: **Latency Score** (lower p95 latency = higher score, logarithmic scale), **Throughput Score** (higher sustainable tokens-per-second = higher score), **CostEfficiency Score** (lower cost per million tokens = higher score, normalized against market median), **Capability Score** (breadth of detected capabilities mapped to a rubric), and **Reliability Score** (historical uptime percentage from health check records). The composite score is a weighted sum:  
$$\text{Composite} = w1 * \text{Latency} + w2 * \text{Throughput} + w3 * \text{CostEfficiency} + w4 * \text{Capability} + w5 * \text{Reliability}$$
where weights default to 0.20 each but are configurable per-deployment. Scores persist to `internal/verifier/persistence/scores.db` (SQLite) with timestamps for trend analysis.

**Phase 6: Ranking** sorts all verified models by composite score descending and assigns rank positions. Ties are broken by latency score (lower wins), then by capability score (higher wins). The ranked list is exposed through `internal/verifier/registry.go` as a thread-safe slice with copy-on-read semantics. The registry also maintains a map from `provider/model-id` tuples to their current rank and score, enabling  $O(1)$  lookups during translation request routing.

**Phase 7: AI Debate Team Selection** configures the multi-model consensus layer used for high-confidence translations. This component, implemented in `internal/verifier/debate.go`, selects the top-N models from the ranked list (default  $N=3$ ) to serve as a "debate team" — each model independently translates the same input, and their outputs are compared for consistency. Divergence triggers automatic re-translation with a confidence warning. This layer is the final consumer of the scoring pipeline's output.

**Phase 8: Server Start** releases the `StartupVerifier.WaitGroup`, allowing the main HTTP/gRPC servers in `cmd/server/main.go` to begin accepting connections. All phases 1-7 must complete before this release; the server literally cannot start with unverified or unranked models in its configuration.

The score adapter pattern, extracted from `internal/services/llmsverifier_score_adapter.go`, serves as the critical bridge between LLMsVerifier's scoring world and HelixTranslate's provider selection logic. This adapter implements a single exported function: `Adapt(scores []verifier.ModelScore) ([]translation.ProviderPreference, error)`. It translates LLMsVerifier's internal score representation — which includes detailed component scores, rank metadata, and capability flags — into HelixTranslate's `ProviderPreference` struct, which carries only the fields the translation engine needs: `ProviderID`, `ModelID`, `Weight`, and `FallbackOrder`. The adapter is the single point of coupling between the two systems, deliberately designed to absorb schema changes on either side without propagating them. When LLMsVerifier adds a new scoring component, only the adapter's weight calculation changes. When HelixTranslate changes its preference format, only the adapter's output struct changes.

### 1.3 Component Architecture and Data Flow

The integration maps components across three repositories: HelixAgent (the source of proven, production LLMsVerifier code), LLMsVerifier (the API and library boundary), and HelixTranslate (the target system). The mapping is not a naive copy-paste operation. Each HelixAgent source file provides a behavioral specification that is consumed through the LLMsVerifier library API and reimplemented in HelixTranslate with domain-specific adaptations for the translation workload.

**TABLE 1: Complete Component Mapping Across Repositories**

| HelixAgent Source File                      | LLMsVerifier API                                                 | HelixTranslate Target                       |
|---------------------------------------------|------------------------------------------------------------------|---------------------------------------------|
| <code>internal/verifier/startup.go</code>   | <code>llmverifier.New(config *Config) (*Verifier, error)</code>  | <code>internal/verifier/d</code>            |
| <code>internal/verifier/scoring.go</code>   | <code>scoring.NewScoringEngine(cfg ScoringConfig) *Engine</code> | <code>internal/verifier/s engine.go</code>  |
| <code>internal/verifier/discovery.go</code> | <code>discovery.NewService(cfg DiscoveryConfig) *Service</code>  | <code>internal/verifier/d service.go</code> |

| HelixAgent Source File                          | LLMsVerifier API                                                   | HelixTranslate Target                 |
|-------------------------------------------------|--------------------------------------------------------------------|---------------------------------------|
| internal/verifier/verification.go               | verifier.Verify(ctx, provider, model) (*VerificationResult, error) | internal/verifier/p                   |
| internal/verifier/registry.go                   | registry.GetRankedModels() []RankedModel                           | internal/verifier/n adapter.go        |
| internal/verifier/health.go                     | health.NewMonitor(cfg HealthConfig) *Monitor                       | internal/verifier/h monitor.go        |
| internal/services/llmsverifier_score_adapter.go | ScoreAdapter.Adapt(scores []ModelScore) []ProviderPreference       | internal/services/llmsverifier_score_ |
| internal/verifier/debate.go                     | debate.NewTeam(cfg DebateConfig) *Team                             | internal/verifier/d team.go           |
| internal/verifier/persistence.go                | persistence.NewStore(path string) (*Store, error)                  | internal/verifier/persistence/store.g |

| HelixAgent Source File                             | LLMsVerifier API                                          | HelixTranslate Target                              |
|----------------------------------------------------|-----------------------------------------------------------|----------------------------------------------------|
| <code>internal/verifier/ratelimit/bucket.go</code> | <code>ratelimit.NewBucket(rate, burst int) *Bucket</code> | <code>internal/verifier/ratelimit/bucket.go</code> |

The data flow through this architecture follows a directed pipeline with clear ownership boundaries. At the leftmost intake, LLMsVerifier's discovery service executes its three-tier poll cycle: primary tier queries provider APIs every 15 minutes, secondary tier scans public registries every 60 minutes, tertiary tier checks community endpoints every 240 minutes. Each poll produces a set of `DiscoveredModel` records containing provider name, model ID, endpoint URL, detected capabilities, and cost metadata. These records flow into the verification pipeline, which executes the eight-step validation sequence for each new or changed model. Successfully verified models receive a `VerificationResult` containing per-step pass/fail status, latency measurements, capability flags, and rate limit parameters.

Verified models then enter the scoring engine, which computes the five component scores and the weighted composite. The scoring engine reads historical performance data from the SQLite persistence store and writes updated scores back after each computation cycle. The scored and ranked model list is then consumed by the score adapter, which translates it into the `ProviderPreference` format that HelixTranslate's existing provider factory understands. This adapter runs on a 5-minute refresh interval by default, ensuring that HelixTranslate's view of available providers tracks LLMsVerifier's current rankings without imposing synchronous coupling.

On the HelixTranslate side, the adapted provider preferences feed into a new verification-aware provider factory located at `pkg/translator/llm/verified_factory.go`. This factory implements the same `ProviderFactory` interface as the existing nine manual providers — `Create(config ProviderConfig) (Provider, error)` — but its implementation queries the LLMsVerifier adapter rather than instantiating hard-coded provider structs. The existing nine providers are not deleted; they are reimplemented as thin wrappers that delegate to the verified factory when LLMsVerifier is enabled, with a feature flag (`--use-llms-verifier`) controlling the routing. This preserves backward compatibility during the transition period and enables rollback without code changes.

**TABLE 2: Before/After Architecture Comparison**

| Dimension             | Before (Current State)                                             | After (Target State)                                         |
|-----------------------|--------------------------------------------------------------------|--------------------------------------------------------------|
| <b>Provider Count</b> | 9 manually configured providers (OpenAI, Anthropic, Vertex, Azure) | 30+ automatically discovered providers via 3-tier discovery, |

| Dimension                      | Before (Current State)                                                                                            | After (Target State)                                                                                                                                                                          |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                | OpenAI, Cohere, Mistral, Together AI, Ollama, Custom)                                                             | with the set growing as new providers enter the market                                                                                                                                        |
| <b>Validation Depth</b>        | No validation; any configured provider is assumed reachable and functional                                        | 8-step verification pipeline: API reachability, authentication, model existence, response format, latency measurement, capability detection, rate limit compliance, error handling validation |
| <b>Scoring System</b>          | None; provider selection is either random, round-robin, or hard-coded priority                                    | 5-component weighted scoring: latency, throughput, cost efficiency, capability breadth, reliability history, producing a composite rank                                                       |
| <b>Configuration Model</b>     | Per-provider API keys, endpoints, and model IDs in configs/translators.yaml; manual edits required for any change | Single configs/verifier.yaml with provider credentials and scoring weights; all model catalogs auto-discovered and kept current                                                               |
| <b>Failure Handling</b>        | Static fallback to secondary provider with manual configuration                                                   | Automatic circuit breaker integration; health monitor removes degraded models; fallback chains adapt based on real-time scores                                                                |
| <b>New Provider Onboarding</b> | Development task: implement provider interface, add factory registration, update config schema, add tests, deploy | Operational task: add API key to verifier.yaml; discovery and verification happen automatically within the next poll cycle                                                                    |
| <b>Model Quality Assurance</b> | Reactive: user complaints drive investigation and provider removal                                                | Proactive: scoring engine continuously evaluates; gating threshold prevents low-scoring models from serving requests                                                                          |
| <b>Multi-Model Consensus</b>   | Not available                                                                                                     | AI Debate Team: top-3 ranked models translate independently; divergence detection triggers confidence warnings and automatic re-translation                                                   |

## 1.4 Implementation Phase Overview

The integration is structured as eight sequential phases spanning sixteen weeks. Phase boundaries are defined by gating criteria, not calendar dates — a phase does not advance until its gating criteria are demonstrably met. This ensures that each layer of the integration is solid before the next layer is built upon it. The sixteen-

week timeline assumes two backend engineers with prior Go experience, one DevOps engineer for deployment pipeline configuration, and part-time frontend capacity for the enterprise UX phase.

**TABLE 3: Master Phase Timeline**

| Phase                              | Weeks | Key Deliverables                                                                                                                                                                                                                                                                                    | Dependencies                                                                  | Gating Criteria                                                                                                                                                                                                  |
|------------------------------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Foundation</b>               | 1–2   | go.mod updates with LLMsVerifier module requirement; configs/verifier.yaml schema definition; internal/verifier/ project structure; CODEOWNERS and governance documentation; CI pipeline extension for verification tests                                                                           | LLMsVerifier v1.x release tag available; Go 1.25.3 toolchain installed        | Module compiles without errors; go vet passes; config schema validates against all known provider types; directory structure follows HelixTranslate conventions                                                  |
| <b>2. Verification Engine</b>      | 3–4   | 8-step verification pipeline implementation in internal/verifier/pipeline.go; LLMsVerifier client wrapper in internal/verifier/client.go; health check integration with existing circuit breaker; provider-specific test vectors for all 9 current providers                                        | Phase 1 completion; API keys for at least 5 providers available in CI secrets | All 8 verification steps execute independently with deterministic pass/fail; each step produces structured output; pipeline completes within seconds per mode; CI; no data races detected by go test -race       |
| <b>3. Scoring Engine</b>           | 5–6   | 5-component scoring engine in internal/verifier/scoring/engine.go; score adapter port in internal/services/llmsverifier_score_adapter.go; SQLite persistence layer in internal/verifier/persistence/store.go; scoring weight configuration in verifier.yaml; score calculation verification harness | Phase 2 completion; historical cost data available for pricing normalization  | Score calculation produce deterministic output for identical inputs; weight adjustments propagate correctly; persistence round-trip verification adapter output matches HelixTranslate ProviderPreference schema |
| <b>4. Discovery &amp; Registry</b> | 7–8   | 3-tier discovery service in internal/verifier/discovery/service.go; in-memory model registry with copy-on-read in internal/verifier/registry/adapter.go; gating threshold enforcement; background poll scheduler; rate limit bucket integration                                                     | Phase 3 completion; network access to provider APIs from staging environment  | Models discovered from all three tiers within configured intervals; registry serves ranked lookups in <1ms; gating threshold correctly filters sub-threshold models; rate limit prevents API quota exhaustion    |
| <b>5. Runtime Integration</b>      | 9–10  | Verification-aware provider factory in pkg/translator/llm/verified_factory.go; selection                                                                                                                                                                                                            | Phase 4 completion; HelixTranslate                                            | End-to-end translation requests completed successfully through                                                                                                                                                   |



| Phase                     | Weeks | Key Deliverables                                                                                                                                                                                                                                                                                                                                                 | Dependencies                                                                 | Gating Criteria                                                                                                                                                                                                             |
|---------------------------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 6. Enterprise UX          | 11–12 | engine with score-based weighting; fallback chain construction from ranked list; feature flag <code>--use-llms-verifier</code> ; backward compatibility layer for existing 9 providers                                                                                                                                                                           | translation test suite passes on main branch                                 | LLMsVerifier-rou providers; fallback chains activate correctly on simul failures; feature f toggle switches between old and n paths without res p99 latency regre <5% compared to baseline                                  |
|                           |       | Model selection UI showing verification status, scores, and ranks; real-time provider health dashboard; batch translation interface with model preference pinning; admin panel for scoring weight adjustment; user-facing model quality indicators                                                                                                               | Phase 5 completion; frontend component library updated; API contracts stable | UX designs review and approved by product team; mo status indicators update within 30 seconds of health changes; admin weight adjustment apply without res accessibility audi passes WCAG 2.                                |
|                           |       | Unit test suite achieving 100% coverage of <code>internal/verifier/</code> ; integration test suite covering all 8 verification steps against live provider sandboxes; chaos engineering challenges (provider failure injection, network partition simulation, rate limit exhaustion); performance benchmark suite establishing latency and throughput baselines | Phase 6 completion; staging environment mirrors production topology          | All unit tests pass –race flag; integration tests p against 5+ live providers; all cha challenges resolv within 60 seconds benchmarks show p99 latency regre >5% and no throughput regres >10%                              |
| 7. Testing & QA           | 13–14 |                                                                                                                                                                                                                                                                                                                                                                  |                                                                              |                                                                                                                                                                                                                             |
| 8. Documentation & Deploy | 15–16 | Complete API reference for all <code>internal/verifier/</code> packages; deployment runbook with rollback procedures; migration guide for existing provider configurations; operational playbooks for incident response; architecture decision records (ADRs) for all design choices; final security review and sign-off                                         | Phase 7 completion; production deployment pipeline configured                | All documentation reviewed and sign off by tech lead a product owner; deployment to sta completes via automated pipelin <10 minutes; roll procedure tested a verified; security review identifies high or critical findings |

**Resource Requirements.** The integration requires API keys for a minimum of 25 providers to exercise the full discovery and verification pipeline. The core set includes: OpenAI (GPT-4o, GPT-4o-mini, GPT-3.5-turbo), Anthropic (Claude 3.5 Sonnet, Claude 3 Haiku, Claude 3 Opus), Google Vertex AI (Gemini 1.5 Pro,

Gemini 1.5 Flash), Azure OpenAI (deployed via organization's Azure subscription), Cohere (Command R, Command R Plus), Mistral (Large, Medium, Small), Together AI (Llama 3.1, Mixtral), AI21 Labs (Jamba), Perplexity, Groq, Fireworks AI, Replicate, Ollama (local endpoint), DeepSeek, Moonshot AI, OpenRouter, and Anyscale. For the testing phase, sandbox or low-quota keys are sufficient — the verification pipeline sends only minimal test prompts, and the discovery tier primarily reads model catalog endpoints rather than generating completions.

Go version alignment is a hard dependency. Both HelixTranslate and LLMsVerifier must compile with Go 1.25.3 to ensure compatible standard library behavior, particularly around the `slices` and `maps` generic functions, `log/slog` structured logging, and `runtime.Pinner` for CGO interactions if local model inference is enabled. The `go.mod` file changes reflect this alignment.

### Code Block 1: Required `go.mod` Changes

```
// go.mod – additions for LLMsVerifier integration
require (
    // ... existing requirements ...

    // LLMsVerifier core library
    github.com/helixagent/llmsverifier v1.4.2

    // LLMsVerifier indirect dependencies (auto-resolved by go mod tidy)
    // github.com/helixagent/llmsverifier/scoring v1.4.2
    // github.com/helixagent/llmsverifier/discovery v1.4.2
    // github.com/helixagent/llmsverifier/verification v1.4.2
)

// Replace directives for local development and staging builds.
// Remove or comment out for production builds that consume
// published tags.
replace github.com/helixagent/llmsverifier => ../llmsverifier

// Verify Go toolchain alignment.
go 1.25.3

toolchain go1.25.3
```

The `replace` directive supports local development workflows where engineers have both repositories checked out side-by-side. In CI and production builds, the directive is removed and the published `v1.4.2` tag is consumed from the module proxy. The `go 1.25.3` directive enforces the minimum Go version; builds attempted with older toolchains fail with a clear version mismatch error.

### Code Block 2: `configs/verifier.yaml` Top-Level Schema

```
# configs/verifier.yaml – LLMsVerifier configuration for
# HelixTranslate
# Schema version: 1.4.2
# Documentation: https://docs.helixagent.dev/llmsverifier/config-
# reference

verifier:
```

```

# Server configuration for the embedded verifier HTTP API
server:
  host: "0.0.0.0"
  port: 8081 # Dedicated port; separate from
             HelixTranslate's main port
  read_timeout: 30s
  write_timeout: 60s

# Discovery tier configuration
discovery:
  primary_poll_interval: 15m # Provider API catalog queries
  secondary_poll_interval: 60m # Public registry scans
                              (HuggingFace, Replicate)
  tertiary_poll_interval: 240m # Community endpoint probes
  max_concurrent_requests: 10 # Per-tier parallelism limit
  request_timeout: 30s
  backoff:
    initial: 1s
    max: 5m
    multiplier: 2.0

# Verification pipeline configuration
verification:
  enabled: true
  timeout_per_step: 30s
  warmup_requests: 10 # For latency measurement step
  max_retries_per_step: 3
  steps:
    - api_reachability
    - authentication
    - model_existence
    - response_format
    - latency_measurement
    - capability_detection
    - rate_limit_compliance
    - error_handling

# Scoring engine configuration
scoring:
  weights:
    latency: 0.20
    throughput: 0.20
    cost_efficiency: 0.20
    capability: 0.25 # Slightly higher for
                     translation workloads
    reliability: 0.15
  gating_threshold: 0.60 # Minimum composite score for
                          model inclusion
  recalculation_interval: 5m
  history_window: 168h # 7 days of health history for
                       reliability score

# Provider API credentials

```

```

# Keys are loaded from environment variables or secrets manager;
# values here are references, not literal keys.
providers:
  openai:
    api_key: "${OPENAI_API_KEY}"
    base_url: "https://api.openai.com/v1"
    default_models: ["gpt-4o", "gpt-4o-mini", "gpt-3.5-turbo"]
  anthropic:
    api_key: "${ANTHROPIC_API_KEY}"
    base_url: "https://api.anthropic.com"
    default_models: ["claude-3-5-sonnet-20241022", "claude-3-
      haiku-20240307"]
  google_vertex:
    api_key: "${GOOGLE_VERTEX_API_KEY}"
    project_id: "${GOOGLE_PROJECT_ID}"
    location: "us-central1"
    default_models: ["gemini-1.5-pro", "gemini-1.5-flash"]
  azure_openai:
    api_key: "${AZURE_OPENAI_API_KEY}"
    endpoint: "${AZURE_OPENAI_ENDPOINT}"
    api_version: "2024-10-21"
  # ... additional providers follow same pattern ...

# Health monitoring and circuit breaker integration
health:
  check_interval: 30s
  failure_threshold: 3           # Consecutive failures before
    marking unhealthy
  recovery_threshold: 2         # Consecutive successes before
    marking healthy
  circuit_breaker:
    timeout: 60s
    half_open_max_calls: 3

# Persistence configuration
persistence:
  store_path: "./data/verifier/scores.db"
  max_size_mb: 512
  backup_interval: 24h

```

The configuration schema is versioned (1.4.2) and validated against an embedded JSON Schema at startup. The providers section uses environment variable references (`${VAR_NAME}`) rather than literal keys, ensuring that credentials never appear in committed configuration files. The `scoring.weights.capability` parameter is set to `0.25` — slightly elevated from the default `0.20` — to reflect the importance of multilingual capability breadth in translation workloads. The `gating_threshold` of `0.60` means that any model scoring below 60% composite is excluded from the provider pool, providing a quality floor that eliminates underperforming models before they reach users.

The sixteen-week timeline concludes with a production deployment that preserves full backward compatibility through the feature flag mechanism. Existing HelixTranslate installations continue to function with their current nine manual providers even after the LLMsVerifier code is deployed. Activation is a runtime

toggle (`--use-llms-verifier`), not a code change or a migration script. This risk-minimizing approach ensures that the integration can be rolled out gradually — first to a staging environment for two weeks, then to a canary production segment for one week, and finally to full production traffic only after all operational metrics are validated against baseline.